

CS 334 — Homework 2 Submission

Nathaniel Schrader — CS 334 Section 2 — Fall 2026

Honor Code Statement:

THIS HOMEWORK IS MY OWN WORK, WRITTEN WITHOUT COPYING FROM OTHER STUDENTS OR DIRECTLY FROM LARGE LANGUAGE MODELS SUCH AS CHATGPT.

Any collaboration or external resources have been properly acknowledged.

Nathaniel Schrader

1 Decision Boundaries (24 pts)

I count a point on the boundary ($\vec{\theta} \cdot \vec{x} + b = 0$) as misclassified.

(a)

i. (4pts) **No.** With $b = 0$, the input $[0, 0, 0]$ gives $\vec{\theta} \cdot \vec{0} = 0$ for any $\vec{\theta}$, so it's always on the boundary and always wrong.

ii. (4pts) **Yes.** $\vec{\theta} = [1, 1, 1]^T$, $b = -2.5$. Then $\vec{\theta} \cdot \vec{x} + b = x_1 + x_2 + x_3 - 2.5$, which is $0.5 > 0$ for $[1, 1, 1]$ and at most $-0.5 < 0$ for every other input.

(b)

Positives are $[1, 1], [-1, -1]$ and negatives are $[-1, 0], [0, 1]$.

i. (4pts) **Possible.** The positives are distance $\sqrt{2}$ from the origin and the negatives are distance 1. Use $r = 1.2$ with inside = negative and outside = positive.

ii. (4pts) **Not possible.** The scores for $[1, 1]$ and $[-1, -1]$ are $a + b$ and $-(a + b)$, which always have opposite signs (or are both 0). So both positives can't be on the positive side.

iii. (4pts) **Possible.** Center $[-0.5, 0.5]$, $s = 2$, inside = negative. Inside means $\max(|x_1 + 0.5|, |x_2 - 0.5|) < 1$. Both negatives give 0.5 (inside), and both positives give 1.5 (outside).

iv. (4pts) **Possible.** $\alpha = 45^\circ$, $s = 2$, inside = negative. After rotating, inside means $\max(|x_1 + x_2|, |x_1 - x_2|)/\sqrt{2} < 1$. The negatives give $1/\sqrt{2} \approx 0.71$ (inside) and the positives give $2/\sqrt{2} \approx 1.41$ (outside).

2 Perceptron Algorithm with Offset (24 pts)

(a) (4pts) Each mistake on point i adds $y^{(i)}\vec{x}^{(i)}$ to $\vec{\theta}$ and $y^{(i)}$ to b , and this happens α_i times. Starting from 0:

$$\vec{\theta} = \sum_{i=1}^n \alpha_i y^{(i)} \vec{x}^{(i)}, \quad b = \sum_{i=1}^n \alpha_i y^{(i)}$$

(b) (4pts) The distance from a point $\vec{p}$ to the boundary is $\frac{\vec{\theta} \cdot \vec{p} + b}{\|\vec{\theta}\|}$: take any $\vec{x}_0$ on the boundary and project $\vec{p} - \vec{x}_0$ onto the unit normal $\vec{\theta}/\|\vec{\theta}\|$, using $\vec{\theta} \cdot \vec{x}_0 = -b$. Plugging in $\vec{p} = \vec{0}$:

$$d = \frac{b}{\|\vec{\theta}\|}$$

(its sign tells you which side of the boundary the origin is on).

(c) (7pts) Done in `perceptron.py`.

(d) (5pts) $\vec{\theta} = [4, 0]^T$, $b = -6$, $\vec{\alpha} = [1, 0, 11, 2, 0, 4]^T$ (18 mistakes).

i	$\vec{x}^{(i)}$	$y^{(i)}$	α_i	$\alpha_i y^{(i)} \vec{x}^{(i)}$	$\alpha_i y^{(i)}$
0	$[-1, -1]$	-1	1	$[1, 1]$	-1
1	$[-1, 1]$	-1	0	$[0, 0]$	0
2	$[1, 1]$	-1	11	$[-11, -11]$	-11
3	$[3, -1]$	+1	2	$[6, -2]$	2
4	$[3, 0]$	+1	0	$[0, 0]$	0
5	$[2, 3]$	+1	4	$[8, 12]$	4
sum:				$[4, 0]$	-6

The sums match (a). The distance from the origin is $-6/4 = -1.5$ (the boundary is the line $x_1 = 1.5$).

(e) (4pts) **Convergence doesn't depend on order.** If the data is separable, the perceptron convergence theorem says it makes at most $(R/\gamma)^2$ mistakes no matter what order the points come in.

The number of mistakes can depend on order. Different orders cause different updates. On our data the given order makes 18 mistakes, but the order 3, 4, 5, 0, 1, 2 makes 17 and ends at $\vec{\theta} = [3, 0]$, $b = -5$.

3 Linear Regression (30 + 22 pts)

3.1 Linear Regression - Optimization Method (30 pts)

(a) (4pts) Done in `linear_regression.py`.

(b) (0pt) Done in `linear_regression.py`.

(c) (9pts) Done in `linear_regression.py`.

(d) (4pts) $M = 1$. For SGD, # iterations counts every single-point update (20 per epoch).

Algorithm	η	θ_0	θ_1	# iterations	Runtime (s)
GD	10^{-4}	0.350874	0.032232	353,622	4.0158
GD	10^{-3}	0.355890	0.023288	52,628	0.5933
GD	10^{-2}	0.357476	0.020459	6,988	0.0786
GD	10^{-1}	0.357979	0.019562	870	0.0098
SGD	10^{-4}	0.356540	0.022053	578,020	1.7593
SGD	10^{-3}	0.357398	0.019849	74,240	0.2251
SGD	10^{-2}	0.354631	0.017647	6,960	0.0213
SGD	10^{-1}	0.313331	-0.009743	1,980	0.0060
Closed form	–	0.358209	0.019152	–	0.0002

(e) (3pts) *Runtime:* SGD was faster than GD at every η , and both got faster as η went up. *Iterations:* SGD actually did about the same or more updates than GD, but each SGD update only uses one point, so each one is much cheaper. *Coefficients:* with small η , GD stops early because the loss changes by less than 10^{-10} per step, so it ends up away from the answer (e.g. $\theta_1 = 0.032$ vs. 0.019 at 10^{-4}). SGD gets closer at small η , but at $\eta = 10^{-1}$ it bounces around and ends far off, at (0.313, -0.010).

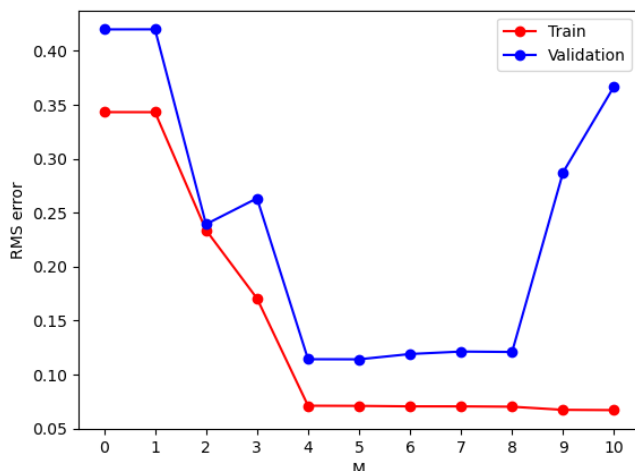
(f) (2pts) The closed form (0.0002 s) was way faster than SGD (0.006 to 1.76 s), since it's just one small matrix inverse. SGD with $\eta = 10^{-3}$ got closest to the closed-form coefficients, within about 10^{-3} .

(g) (3pts code + 5pts written) I used $\eta_k = \frac{0.2}{1 + k/100}$, where k is the number of updates so far. It takes big steps at the start and smaller steps later, so SGD stops bouncing around. It converged in 25,140 updates (0.079 s) to $\vec{\theta} = (0.358215, 0.018545)$, which is closer to the closed form than any fixed η . I picked 0.2 and 100 by trying a few values.

3.2 Linear Regression - Overfitting & Regularization (22 pts)

(a) (0pt) Done in `linear_regression.py`.

(b) (5pts)

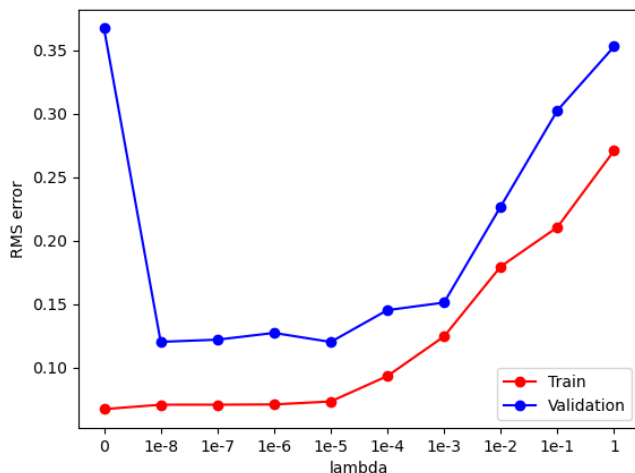


M	Train	Validation
0	0.3433	0.4200
1	0.3432	0.4200
2	0.2339	0.2394
3	0.1705	0.2634
4	0.0711	0.1142
5	0.0710	0.1141
6	0.0705	0.1191
7	0.0705	0.1213
8	0.0702	0.1209
9	0.0673	0.2875
10	0.0670	0.3671

(c) (3pts) $M = 4$ fits best. $M = 5$ is only lower by 0.0001, so I'd pick the simpler model. $M \leq 3$ underfits, since both errors are high. $M \geq 9$ overfits: training error keeps going down but validation error jumps up to 0.29 and 0.37.

(d) (3pts) Done in `linear_regression.py`: $\vec{\theta} = (\Phi^T \Phi + \lambda I)^{-1} \Phi^T \vec{y}$.

(e) (5pts) $M = 10$. I spaced the λ values evenly on the x-axis since 0 can't go on a log scale.



λ	Train	Validation
0	0.0670	0.3671
10^{-8}	0.0705	0.1201
10^{-7}	0.0705	0.1218
10^{-6}	0.0707	0.1271
10^{-5}	0.0731	0.1200
10^{-4}	0.0931	0.1451
10^{-3}	0.1243	0.1510
10^{-2}	0.1795	0.2268
10^{-1}	0.2104	0.3024
10^0	0.2707	0.3528

(f) (3pts) $\lambda = 10^{-5}$ is best (validation 0.1200), though 10^{-8} is basically tied. With $\lambda = 0$ the model overfits (0.367). Adding even a tiny λ cuts validation error to about a third. For $\lambda \geq 10^{-4}$ both errors go up, so the model starts underfitting.

(g) (3pts) I'd pick $M = 4$ **with no regularization**. It has the lowest validation error (0.114 vs. 0.120) and it's a simpler model. But with only 20 training and 20 validation points the differences are small and could just be noise, so before deciding I'd use cross-validation over different (M, λ) pairs and then check the final model on a separate test set.